Plexus

A blueprint for differentiable tissue

Janelia Research Campus

August 24, 2026

Abstract

We introduce *Plexus*, a typed intermediate representation for biological models. Rather than defining yet another simulator, Plexus defines a common language that decomposes a biological system along two axes at once: *what exists* — heterogeneous entities, the states they carry, the relations between them and the levels that organize them — and *what happens* — the mechanisms acting on them. Neither axis is subordinate to the other. A mechanism such as adhesion has no meaning apart from what is adhering to what, and a model of a tissue, its basement membrane and the surrounding matrix is informative precisely because those remain distinct kinds of entity rather than being flattened into variables of one universal object. Every operator therefore declares the entities it acts upon, the states it requires, the relations it needs, and one or more numerical implementations, while a hierarchy of levels keeps heterogeneous entities, fields and relations organized rather than merged. Decomposition serves recomposition: the purpose is to take existing models apart into reusable pieces and to build new biological compositions from them, which in turn requires the language that specifies a model to be separated from the interpreter that executes it. The Plexus research program has three objectives. First, to define an algebra of biological operators over these organized entities. Second, to decompose the existing corpus of simulation models into this common language. Third, to construct differentiable implementations of these operators so that mechanistic models can be directly inferred from biological data.

1 Introduction

Computational biology has produced a rich ecosystem of simulation frameworks. Neural simulators model electrical activity, reaction–diffusion systems model chemical transport, particle and continuum methods model mechanics, while vertex, Cellular Potts and active-matter models describe multicellular tissues. These frameworks remain however largely incompatible. Combining electrical signalling, mechanics, metabolism and development often requires coupling multiple specialized simulators whose underlying abstractions differ fundamentally. Each simulator introduces its own representation of biological mechanisms, making models difficult to compare, compose and extend. This fragmentation is not primarily a numerical problem but more a language problem.

Instead of treating simulation frameworks as fundamental, we treat the biological system itself as the object to be decomposed. A biological system must be decomposed not only into mechanisms, but also into the entities those mechanisms act upon. These entities are heterogeneous — receptors, signalling proteins, cells, extracellular matrices, tissues, fields — and they are organized across biological levels. Plexus makes that organization explicit as a compositional hierarchy: each level carries its own entities and states, operators act between entities within a level, and Aggregate and Broadcast connect levels by mapping state upward and downward. Decomposition therefore does not flatten biology into a single collection of variables; it preserves the organization through which mechanisms at one level are composed with mechanisms at another.

On the mechanism side, Plexus introduces biological operators: adhesion, diffusion, division, chemotaxis, electrical propagation or active stress, described independently of their numerical implementation. Entities and operators are the two halves of one decomposition, and neither is primary. An operator is typed by the entities, states and relations it requires, and a set of entities is inert until operators act on it.

The purpose of this decomposition is ultimately recomposition: to construct complete mechanistic models from reusable entities and operators, rather than merely to classify the mechanisms of existing simulators. Making such compositions executable requires a strong structural choice — the language that specifies a biological model must be separated from the code that interprets and executes it. The language declares entities, states, relations, hierarchy and operators; the interpreter resolves that description into concrete numerical implementations and schedules their execution. In this perspective, biological modelling becomes analogous to compiler design: just as modern compilers separate programming languages from their implementations through an intermediate representation, Plexus separates biological mechanisms and their organization from the numerical methods that realize them. The analogy is a consequence of wanting free recomposition, not the premise it starts from.

This viewpoint defines a broader research program. First, Plexus seeks to construct an algebra of biological operators whose elements can be composed into complex dynamical systems, compared through their observable consequences, and expanded as new mechanisms are discovered. Second, Plexus aims to extract this operator algebra from the existing corpus of biological simulation models. Rather than viewing current frameworks as competing simulators, we regard them as repositories of reusable mechanisms. By decomposing these models into a common language, we seek to identify a compact vocabulary capable of expressing a wide range of biological dynamics across scales. Third, Plexus constructs differentiable counterparts of these operator compositions. Differentiability allows mechanistic models to be fitted directly to biological data, residuals to be interpreted mechanistically, and missing operators intuited.

Finally, because biological mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

2 The language

Plexus defines a typed language for expressing biological models independently of their numerical realization. The language names *what exists* — the *sets* of entities, the *states* they carry, the *fields* they live in, the *relations* between them and the *hierarchy* that organizes them — and *what happens* — the *operators* that transform them. Biological models are therefore represented as compositions of operators acting on organized discrete and continuous biological objects rather than as simulator-specific algorithms. The following subsections take these in turn: operators, sets, states, fields, and the hierarchy that holds them.

2.1 Operators

Operators are the fundamental objects of the language. An operator represents a single biological mechanism, such as adhesion, chemotaxis, diffusion, electrical propagation, active stress, metabolism, growth, or cell division. Every operator is defined by a typed signature and one or more numerical implementations. The signature specifies *what* biological transformation is performed, whereas implementations specify *how* it is numerically realized. The signature specifies

- the input sets,
- the output sets,
- the state variables consumed and produced,
- the maps relating the participating sets.

Maps define the relations required by the mechanism. They determine, for example, which particule belong to which cell, which synapses connect which neurons, or how cells relate to a continuous field. Plexus incorporates maps into the operator, making operators typed morphisms between biological sets.

2.2 Sets

Sets represent collections of biological entities. Examples include genes, proteins, cells, neurons, synapses, tissues, organs and organisms. A set is homogeneous in kind: its elements share one state schema and one biological interpretation.

Sets organize into hierarchies reflecting biological organization, but a level of that hierarchy is *not* a single set, and this is the point at which a familiar simplification has to be resisted. Writing the hierarchy as a chain $S_0 \to S_1 \to S_2 \to S_3$ — molecules, then cells, then tissues, then organism — suggests one biological category per level. Biology is not so uniform. A cell contains receptors, signalling proteins, cytoskeletal elements and, in a mechanical model, material points; a tissue contains cells, a basement membrane and an extracellular matrix. Each of those is a distinct set with its own state and its own operators, and all of them share a parent. A level is therefore a heterogeneous collection

$$L_k = \{\, S_k^1, S_k^2, \dots,\; F_k^1, F_k^2, \dots,\; R_k^1, R_k^2, \dots \},$$

of sets S_k^i, fields F_k^j bound to that level, and relations R_k^l among them. The hierarchy organizes these objects; it does not define them by a single molecule/cell/tissue label. Each level is expressed in the same language, allowing intracellular signalling, multicellular mechanics, developmental processes and organ-level physiology to be described without translation between formalisms.

2.3 States

Every element of a set carries a state describing its current biological condition. While sets define *what* biological entities exist, states define *what each entity carries*. The state schema is specific to each set. A protein may carry position, binding state and the load it bears; a material point position, velocity and deformation gradient; a neuron membrane voltage and calcium; a cell polarity, stiffness and gene-expression levels; a synapse conductance and plastic weight. Operators transform states. They consume selected state variables and produce updates to others, while leaving the underlying biological entities unchanged.

2.4 Fields

Fields represent continuous quantities defined over space rather than over discrete biological entities. Examples include morphogen concentrations, pressure, extracellular signalling molecules, electric potentials, and material properties. Fields complement sets. While sets describe discrete biological entities, fields describe the continuous environment in which these entities evolve. Operators couple sets and fields through explicit maps relating discrete entities to continuous spatial representations.

2.5 Hierarchy

Containment is declared, not implied. A set may be contained in another — proteins in a cell, cells in a tissue, tissues in an organism — through an explicit map π from children to parent. A parent may contain several distinct sets at once: the same cell can hold receptors, signalling proteins, cytoskeletal elements and material points, each remaining a distinct object with its own state and its own operators while sharing that parent. Every level carries its own state, its own operators and its own rate, so no level is reduced to a parameter of the level above it.

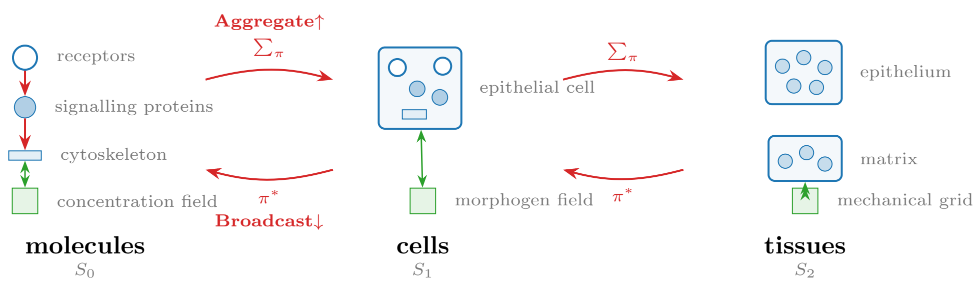

Figure 1: The hierarchy. Each level holds several *distinct* sets (blue) and may carry its own fields (green); operators (red) act between entities within a level, and exactly two families cross a level — Aggregate, $\sum_\pi$, and Broadcast, π^*.

Operators act within a level or across levels. Within a cell, operators may couple receptors to signalling proteins to cytoskeleton. Across the containment map, exactly two families apply: **Aggregate** builds parent state from children — a cell position from its material points, a neuron input from its synapses — and **Broadcast** propagates parent state to children — a material property to every point a cell owns, or a constraint imposed by the parent. Formally,

$$L_k \xrightarrow{\text{Aggregate}} L_{k+1}, \qquad L_{k+1} \xrightarrow{\text{Broadcast}} L_k.$$

Fields are level-specific in the same way. Exchange couples a set to a field at its own level, so a molecular concentration field, a cellular morphogen field and a tissue-scale mechanical grid coexist as distinct objects rather than being merged into one grid.

The consequence is that molecules, cells and tissues are not a pipeline of separate simulators handing results to one another. They are one compositional graph: several sets may coexist inside each parent, operators connect them locally, and Aggregate and Broadcast connect the levels. The same operator families are reused throughout, so adding a biological scale adds no machinery to the engine.

3 The operator algebra

The language introduced in the previous section defines the objects manipulated by Plexus. We now define the elementary operator families from which biological models are constructed. The central hypothesis of Plexus is that a large fraction of existing biological simulation models can be expressed as compositions of a small number of elementary operator families. Rather than introducing a new operator for every biological phenomenon, Plexus identifies a compact operator algebra that can be reused across mechanics, signalling, metabolism, development and physiology.

The proposed operator algebra consists of eight elementary families (Fig. 2):

- **Lateral**: interactions within a biological set.
- **Aggregate**: many-to-one transformations across biological scales.
- **Broadcast**: one-to-many transformations across biological scales.
- **Exchange**: transformations between sets and fields.
- **Rewire**: modification of relations between biological entities.
- **Divide**: creation of biological entities.
- **Die**: removal of biological entities.
- **Seed**: establishment of the initial state.

The first four families transform biological state while preserving the underlying biological organization. Rewire modifies the interaction graph, whereas Divide and Die modify the biological entities themselves. Seed writes the state a model starts from. Together they form the elementary computational vocabulary of Plexus.

Seed is separated from Divide and Die — with which it shares a mechanism, since all three write state directly rather than returning a delta — because it differs in *when*. Divide and Die act throughout a trajectory; Seed acts only at its opening. Treating that distinction as a naming convention rather than a kind has a concrete cost. In an automated model-search campaign on a growing epithelial vesicle, an initial condition for a reaction–diffusion field was re-applied on every timestep rather than once, which converts it from an initial condition into a moving boundary condition and silently overwrites the state of any operator writing to the same channel. Both spellings were legal specifications and nothing in the language could distinguish them; the resulting runs reported a mechanism as refuted eight times across thirteen rounds when the operator under test had in fact never reached the state. Because Seed is a kind, the runtime imposes its window, and a per-timestep initial condition is not expressible.

Lateral. Lateral operators describe interactions between entities belonging to the same set. Typical examples include mechanical forces between proteins, adhesion between neighbouring cells, electrical communication between neurons, or biochemical interactions within a metabolic network.

Aggregate. Aggregate operators propagate information from many entities to a higher level of biological organization. Examples include computing cellular properties from their constituent proteins or accumulating synaptic currents acting on a neuron.

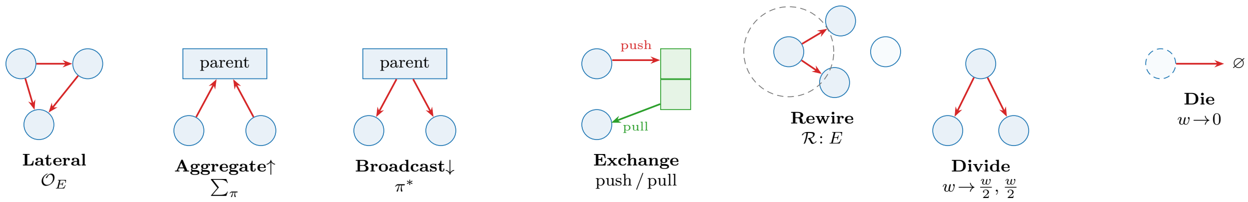

Figure 2: The elementary operator algebra of Plexus. Lateral, Aggregate, Broadcast and Exchange transform biological state. Rewire modifies biological relations, whereas Divide and Die modify biological entities. Complex biological models are obtained by composing these elementary operator families.

Broadcast. Broadcast operators perform the complementary transformation, propagating information from higher organizational levels toward their constituents. Aggregate and Broadcast therefore define the upward and downward flow of information across biological scales.

Exchange. Exchange operators couple discrete biological entities with continuous fields. Typical examples include secretion, sensing, transport, uptake, or particle–grid transfer.

Rewire. Rewire operators modify the relations between biological entities without changing the entities themselves. They update neighbour graphs, contact networks, developmental connectivity or other interaction structures, determining which entities participate in subsequent operator evaluations.

Divide and Die. Living systems continuously modify their own composition. Divide creates new biological entities, whereas Die removes existing ones. These operators modify the biological organization itself rather than only its state.

4 Composition

A biological model is built by composition, and composition happens in four distinct ways. Each answers a different question, and a model is complete only when all four are answered.

1. **Entities compose into structures.** Sets, the states they carry and the relations declared between them form the objects a model is about: a membrane is a set of faces with an incidence relation, a connectome is a neuron set with an edge-set over it, a tissue is a cell set with a containment map to its molecules. This composition is structural and is fixed by the specification before any dynamics run.

2. **Mechanisms compose within a level.** Several operators may act on the same set, each contributing its own increment: adhesion and active stress on one cell set, diffusion and reaction on one chemical field. Because operators are typed by what they read and write rather than by the simulator they came from, mechanisms from different modelling traditions may act side by side on the same entities.

3. **Levels compose through Aggregate and Broadcast.** A mechanism written at one level reaches another only through the containment map of Section 2.5, $L_k \to L_{k+1}$ upward and $L_{k+1} \to L_k$ downward. This is what allows a molecular mechanism and a tissue-scale mechanism to belong to one model without either being rewritten in the other's terms.

4. **Mechanisms compose in time.** The three compositions above say what acts on what; the *schedule* says in which order and how often.

The first three are the subject of the preceding sections. The remainder of this section formalizes the fourth.

Rather than embedding all mechanisms within a monolithic simulator, Plexus represents a model as an ordered sequence of operator applications. This sequence, called the *schedule*, specifies both the order and the frequency with which operators are evaluated. Formally, one simulation step is the composition

$$\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1,$$

whose repeated application generates the system dynamics

$$x_{t+1} = \Phi(x_t).$$

The schedule therefore describes the temporal organization of biological mechanisms. Different operators may evolve on different time scales. Mechanical interactions may be evaluated every simulation step, whereas diffusion, growth or cell division may occur less frequently. A biological model therefore becomes a multi-rate composition of biological mechanisms rather than a monolithic numerical solver.

Operators are pure transformations. Rather than modifying biological state directly, every operator returns its contribution to the system evolution. If operator $\mathcal{O}_i$ produces the increment Δ_i, the complete update is obtained by

$$\Delta = \sum_i \Delta_i,$$

before being passed to the numerical implementation. This formulation has two important consequences. First, biological mechanisms remain independent and composable: introducing a new mechanism simply requires adding a new operator to the schedule. Second, the language remains independent of its numerical realization. The schedule specifies the biological model, whereas the numerical implementation specifies how that model is evaluated.

5 Numerical implementations

The operator algebra defines the biological semantics of a model independently of its numerical realization. Every operator may admit multiple implementations satisfying the same typed signature. These implementations differ in numerical realization while preserving identical biological semantics. Selecting an implementation is therefore a numerical decision rather than a biological one.

This abstraction naturally accommodates a broad range of computational methods. Mechanical operators may be implemented using Material Point Methods, finite elements, vertex models, phase-field methods, Cellular Potts models or optimal-transport formulations. Signalling operators may employ Hodgkin–Huxley equations, graph neural networks or Neural ODEs. Likewise, field operators may use finite differences, finite elements, neural implicit representations or spectral methods. These implementations coexist within the same language because they satisfy identical operator contracts.

Different implementations may also employ different numerical integration schemes. Depending on the operator, implementations may evolve first-order states such as concentrations or membrane voltages, second-order states such as mechanical accelerations, or rely on explicit, implicit or adaptive solvers. These choices affect computational efficiency and numerical accuracy but not the biological interpretation of the model.

This distinction fundamentally changes how biological simulations are compared. Traditional frameworks compare complete simulators, intertwining biological mechanisms with numerical algorithms. Plexus instead separates these two questions. Biological models are compared through the operators they compose, whereas numerical methods are compared through the implementations realizing those operators.

Because biological mechanisms are represented in a formal language rather than embedded in simulator-specific code, they become amenable to automated reasoning. The explicit separation between biological semantics and numerical implementations allows agentic systems to operate directly on the language itself, composing, comparing and optimizing mechanistic models without modifying simulator code. The search for biological explanations therefore becomes a search over model structures rather than simulator implementations.

6 Mechanistic inverse modelling

One of the primary motivations for introducing a formal language of biological mechanisms is to identify mechanistic models directly from experimental observations. Rather than calibrating simulations manually, the objective is to infer the parameters, spatial organization and constitutive laws governing biological systems from imaging data.

Plexus makes this possible by associating every biological operator with one or more differentiable implementations satisfying the same typed signature. Differentiability therefore becomes a property of the language rather than of a particular simulator. Regardless of the numerical backend, gradients propagate through compositions of explicit biological operators rather than through monolithic simulation code.

This distinction fundamentally changes the role of inverse modelling. Traditional approaches assume that the underlying mechanism is already correct and seek the parameter values that best reproduce the observations. Such parameter optimization is effective when the forward model is complete, but it becomes misleading when important biological mechanisms are absent. Optimizing existing parameters may improve the fit while merely compensating for missing mechanisms.

In Plexus, parameters remain attached to explicit biological operators rather than to opaque numerical models. Mechanical properties belong to mechanical operators, reaction rates to biochemical operators, conductances to signalling operators, and spatial fields to the mechanisms generating them. Parameter estimation therefore preserves the mechanistic interpretation of the model.

Residuals likewise acquire a mechanistic interpretation. A persistent mismatch between simulation and experiment no longer indicates only poor parameter values; it may also indicate that the current operator composition is unable to explain the observations. Differentiability thus provides more than efficient optimization: it enables quantitative comparison between competing mechanistic models while preserving their explicit biological structure.

Crucially, because the operator composition is differentiable end to end, the objective need not span the entire system: a loss may be *local* in both space and time. It can be evaluated on a chosen *subset of entities* — a region of interest such as the extrusion front where activated and quiescent cells meet — over a *short rollout* of a few tens of steps, rather than on every cell across the whole trajectory. Gradients still propagate through the shared operator composition, so a *local* objective fine-tunes precisely the mechanism responsible for a local behaviour — the shape of a growth front, the sharpness of a domain boundary, the timing of a rearrangement — without the averaging-out of a global fit and, decisively, without the prohibitive cost of differentiating a long, whole-system rollout. This last point is what makes the approach practical at all: a loss spanning every cell over hundreds of frames unrolls, under reverse-mode differentiation, into a backpropagation graph whose memory and compute scale as cells $\times$ steps and quickly become intractable, whereas a loss confined to a small set of entities over a short window keeps that gradient graph small. *Locality* is therefore the enabling property of differentiable refinement on large, long-lived tissues: the mechanism is global and shared, but the *evidence* constraining any single operator is usually confined to a small, transient region, and the loss should be placed exactly there.

7 Agentic mechanistic discovery

> *"What I cannot create, I do not understand."*
> *"Know how to solve every problem that has been solved."*
>
> — Richard P. Feynman[1]

Feynman's two lines frame this section. A biological mechanism is understood only when an agent can *create* it — assemble it from operators until it reproduces the observations — and new discovery begins only once the agent can already re-derive every model that has been solved. The three loops below make this operational, moving from reconstructing what is already known toward constructing the mechanisms that existing models cannot yet express.

The explicit representation of biological mechanisms transforms scientific discovery from a parameter optimization problem into a language exploration problem. Because mechanisms are represented as explicit operators rather than embedded in simulator-specific code, they become directly accessible to agentic systems.

Plexus proposes a three-loop strategy for mechanistic discovery. Each loop operates at a different level of abstraction, progressively moving from existing models toward new biological knowledge.

Loop I: Understanding. The first loop takes an existing model — a paper and its repository — and decomposes it into Plexus operators, sets, states and fields, so the biological mechanisms it implements become explicit. That decomposition is necessary, but it is only the *entry*: it makes the model's *levers* explicit without yet explaining them. Understanding is earned afterwards, by *interrogating* the decomposition as a causal system — learning what each operator and parameter does on its own and, the harder part, what it does in combination, since mixtures rarely surrender their causal structure to inspection. The driving force is therefore not translation but a *hypothesis* $\rightarrow$

[1] Found written on Feynman's blackboard at Caltech at his death in February 1988. Never published in a paper or lecture, it survives as a bare aphorism; the second line stood directly beneath the first on the same board. Photographs of the blackboard are widely reproduced.

validate/falsify cycle run on the decomposed model: each step states a falsifiable claim about a lever, tests it against a dedicated metric, and accumulates the causal map that results. A useful operating point is a rough **70/30 balance of validation to falsification**: pure validation only ever confirms hypotheses one already believes (trivial, near-zero information), while pure falsification only ever probes the improbable (which mostly breaks as expected and never consolidates a working map) — so most tests should confirm plausible levers while a deliberate minority attacks the current understanding, where a surprise is most informative. The loop's product is a causal lever-map of the model — what each mechanism buys, and how mechanisms combine — which is what makes the operators genuinely *reusable* rather than merely catalogued, and what later tells Loop III which lever the model *lacks*.

Loop II: Fitting. The second loop estimates the parameters, spatial fields and initial conditions of the resulting mechanistic model by fitting it directly to experimental observations. Because every operator admits a differentiable implementation, optimization preserves an explicit correspondence between parameters and biological mechanisms.

Loop III: Discovery. The third loop analyses the residual remaining after optimization. Persistent discrepancies are interpreted as evidence that the current operator composition is incomplete rather than simply poorly parameterized. Turning Loop I's hypothesis-driven validate/falsify cycle outward — from the levers the model *has* to the ones it *lacks* — the agent proposes candidate operators, compares competing mechanistic hypotheses, and extends the operator algebra whenever existing mechanisms fail to explain the observations.

Together, these three loops transform simulation, inverse modelling and hypothesis generation into successive stages of a single scientific workflow. The language itself becomes the object of discovery: new biological knowledge is represented by extending the operator algebra rather than merely adjusting parameter values.

Orchestration: loop agents and a master controller. The three loops are realized as autonomous agents coordinated by a *master*: each performs one kind of work — Understanding, Fitting, or Discovery — and communicates only through typed artifacts on a shared blackboard (the mechanistic model, the success metrics with current values, the residual, and a queue of falsifiable hypotheses). The master never performs the loops' work; it is a state machine whose guards are the metrics — advancing a stage when its metric is met, invoking Discovery when a residual persists, re-invoking Fitting once a new operator is admitted. Beyond scheduling it carries a few responsibilities: it *authors* the success metrics, so "goal achieved" is a falsifiable test rather than a visual impression; it *grounds* hypotheses in the reference literature and code before Discovery proposes; it gates every proposal on *compliance* (a well-typed Plexus contract, not a simulator-specific patch) and *consistency* (an active search for violated invariants — determinism, conservation, agreement between overlapping runs — that separates "the model is incomplete" from "the measurement is wrong"); and it holds each stage open until settled, refusing the "good enough to move on" shortcut.

The master reproduces a modeller's discipline. These responsibilities are not invented for the machine; they formalise how a careful modeller works, distilled from a prototype that deciphers the `tyssue` epithelial vertex model into Plexus operators.[2] Four disciplines recur.

[2]Reported separately as a living document of dated field notes: it reconstructs the model as mechanism-named contracts and works each morphogenesis stage by the method below.

- **Verify the instrument before trusting the measurement** — the sharpest, most-repeated lesson: a "97% hollow" shell that was a recording-stride *misalignment* (vertex positions paired with another frame's connectivity, caught by a determinism contradiction — a long run diverging from its own shorter prefix), and a false "0 invalid faces" that passed self-intersecting bow-tie polygons because the guard checked area > 0, not a simple embedded tiling. An agent that skips this gate elaborates operators to explain an artefact.
- **Small, falsifiable, metric-gated steps — one factor at a time, cheap-first.** Loop I's validate/falsify cadence, enforced across every stage: isolating proliferation rate, division timing and volume stiffness is what homogenised the tissue; a blind sweep would not attribute the win.
- **Perfecting.** A satisfied metric is not qualitative fidelity: the triangular-versus-round-cells gap became a newly authored metric (shape index, sliver fraction) and a target to close.
- **A persistent, revisable master memory** — a log of hypotheses validated and overturned, grounding reads from the reference literature, and the operator atlas — so the search accumulates knowledge instead of re-deriving it each round.

What the loop cannot yet supply — and the agents that must. A prototype run with a human in the loop makes the division of labour precise, and honest about its frontier: the worker agents are strong at execution, breadth and bookkeeping, but four contributions came reliably from the human and name what the scheme must still internalise.

- **Goal custody** — the single most valuable intervention. A worker elaborates whatever it is touching and cannot catch its own drift ("I thought we were starting tubing from 2000 cells"); the master's objective must be held *separately* from the workers'.
- **Temporal perception** — a genuinely structural gap: an agent reads a static strip but cannot *watch a movie*, yet the decisive evidence was in the time course (a pattern that forms only at 12 s; a spot that decays then reforms; tip cells that balloon at 2 s). A dynamics-watching critic is a required agent, not a convenience.
- **Domain priors and taste** — reading the activator gradient off Okuda's figure; naming amplification feedback before finding it is Gierer–Meinhardt.
- **Literature grounding** — "which reaction–diffusion regime does Okuda use?" $\rightarrow$ the GM unlock.

A faithful multi-agent scheme is therefore not one master and identical workers but a small division of labour — a goal-custody supervisor, a dynamics-watching critic, and a literature-grounding reader — around the disciplined loops above.

A Reference implementation

The main text defines Plexus as a formal language for expressing biological mechanisms. This appendix describes the reference implementation of that language. The implementation follows the abstractions introduced in the paper without introducing additional biological concepts. Its purpose is simply to execute models written in the Plexus language.

The implementation distinguishes two levels. The *language* defines the biological semantics through operators, sets, states and fields. The *runtime* provides concrete software classes implementing these concepts.

Language	Reference implementation
Operator (contract)	`OperatorContract`
Implementation	an `Operator` subclass, registered under a contract
Set	`Level`
State	typed tensors stored by a `Level` (sliced by its `state_schema`)
Map	an index buffer on a `Level` (`parent`, `pre`, `post`)
Field	`Field`
Biological model	`Hierarchy`
Registry	`@register_operator` / `@register_entity` / `@register_field`

Maps are not a separate runtime object. Consistently with the operator-centric view of the main text, a map is an index buffer carried by a `Level` (`parent`/`parent_name` for containment, `pre`/`post` for incidence) and *named* by the operators that traverse it in their typed signature — so the runtime realizes four primitives (operators, sets, states, fields), with maps folded into the operator.

A.1 Hierarchy

The `Hierarchy` represents one executable biological model. It is the runtime container responsible for assembling the language primitives into a simulation.

A Hierarchy stores

- the collection of biological Levels,
- the continuous Fields,
- the instantiated Operators,
- the execution Schedule,
- backend-specific runtime data.

The Hierarchy therefore corresponds to one complete biological model expressed in the Plexus language.

A.2 Levels

A `Level` is the runtime representation of a biological Set. Its elements are stored as flat batched tensors (never one object per element).

Concretely, a Level carries

- a `state` tensor and the `state_schema` that names its columns (`pos`, `vel`, `voltage`, ...);
- an integer `depth` (a scale hint; 0 for a leaf set);
- optional biological `types` (parametric variants within the set);
- its maps, as index buffers: `parent` (with `parent_name`) for containment, and `pre`/`post` for the incidence of an edge-set;

- an occupancy vector occ$\in [0,1]$ marking the live subset, so a cardinality-changing operator (Divide/Die) wakes or retires fixed slots without resizing.

State is stored as contiguous tensors for computational efficiency, while its biological interpretation is determined entirely by the `state_schema` declared by the language. Consequently, the runtime never assumes that a particular state variable represents position, voltage, concentration or any other specific quantity: operators read named blocks (`lvl.get("pos")`) rather than hard-coded column offsets.

A.3 Fields

Fields represent continuous spatial quantities. Unlike Levels, which contain discrete biological entities, Fields describe continuous media such as morphogen concentrations, extracellular signals, pressure or material properties.

Operators interact with Fields exclusively through the maps declared by their typed signatures. The numerical realization of these maps (interpolation, particle–grid transfer, neural implicit evaluation, etc.) is entirely implementation dependent.

A.4 Operators

The runtime realizes the contract/implementation split of Section 5 directly. A biological operator is an `OperatorContract` — a fixed record of `(name, kind, family, set, signature)` together with a dictionary of interchangeable `implementations` and a `default`. Each implementation is an `Operator` subclass; the contract owns the biology, the subclass owns only the numerics.

An operator is registered by decorating its implementation class. The FIRST registration of a name creates the contract from the class's typed `signature()`; a later registration of the same name with a different `implementation=` adds an interchangeable implementation to that same contract (the engine checks the two share a `kind`, so only the numerics may differ).

Listing 1: An operator implementation: the decorator names the contract (`set`/`kind`/`family`) and this implementation; the class attributes are the typed signature and the capabilities.

```
@register_operator("neuron_voltage", set="neuron", kind="lateral",
                   family="interaction", implementation="hodgkin_huxley")
class HodgkinHuxley(Lateral):
    # typed signature -- the biology (Plexus2 sec. 2.1)
    INPUTS  = ["neuron", "synapse"]   # input sets
    OUTPUTS = ["neuron"]              # output sets
    READS   = ["voltage", "w"]        # state consumed
    WRITES  = ["voltage"]             # state produced
    MAPS    = ["pre", "post"]         # maps traversed (folded into the operator)
    # capabilities -- the numerics
    EMIT           = "velocity"       # integration order of the returned delta
    SUPPORTED_DIMS = [2, 3]
    DIFFERENTIABLE = True
    def forward(self, H, mask=None):
        ...                           # returns {"neuron": d_voltage}
```

Three things follow from this shape.

Signature. `signature()` returns the typed morphism {`inputs, outputs, reads, writes, maps`}, defaulting the sets to the acting set `SET` when a class leaves them unset. This is the machine-readable form of "operator as a typed morphism between sets"; the validator and the atlas read it, the engine never does.

Capabilities. Each implementation declares `SUPPORTED_DIMS` and `DIFFERENTIABLE`; the contract's `capabilities()` tabulates them across implementations, so an inverse loop can select, e.g., the differentiable implementation of a contract.

Kind and purity. Every operator belongs to exactly one `KIND` — the eight of the main text (`lateral`, `aggregate`, `broadcast`, `exchange`, `field`, `rewire`, `structural`, `seed`). Dynamics operators are pure: `forward` returns a per-set delta and the engine integrates once per tick (its order set by `EMIT` $\in$ {`velocity`, `acceleration`, `mpm_acceleration`}); `field/rewire/structural/seed` operators mutate the field, the relation, the membership, or the initial state in place and return nothing.

The seed window. `seed` is the one kind whose *schedule* is part of its contract rather than of the specification: the runtime reads the specification once before execution and confines every seed to the opening frames. The window is the largest one any seed in that specification requests, defaulting to a single frame and capped at a small maximum. A model may therefore establish its initial state over the few frames it needs, for instance while a mesh relaxes, but cannot ask an initial condition to run for the length of the trajectory. This is the same move as `EMIT`: a discipline every specification would otherwise have to remember becomes a guarantee the language provides.

Model and implementation are different choices. A contract may be varied along two axes, and conflating them makes a model-search campaign unable to say what it found.

An **implementation** is a numerical realization of a fixed biology. Its variants share the contract's parameters and answer the same question with different numerics: diffusion on a combinatorial graph Laplacian or on interface-weighted finite volumes; a finite-difference or a spectral field update. Swapping one for another holds the operating point fixed, and a result that changes under the swap is a statement about discretization, not about the tissue.

A **model** is a different biological hypothesis occupying the same slot. Gray–Scott, Brusselator and Gierer–Meinhardt are all reaction operators on a two-species state, but they are not three ways of computing one reaction: their parameter sets are disjoint ({`F`,`kk`}, {`A`,`B`}, {`a0`,`mu_a`,`sat`}), so no common operating point exists at which to compare them. Likewise an operator coupling shape back to chemistry may sense curvature, cortical tension, apical area or pressure; these share a signature exactly, yet each is a distinct claim about mechanotransduction, resting on different evidence. Swapping a model *is* an experiment.

Declaring both through one keyword had a concrete cost. In our developmental campaign the search's own documentation described implementation swaps as "genuine mechanism edits" while this appendix described them as numerical choices; every finding recorded against the reaction operator was in fact a finding about Gray–Scott, and the search verb that was pushed hardest by coverage analysis was the one whose meaning was ambiguous. The registry's only guard — that variants of a contract share a `KIND` — cannot detect this, since all four shape-sensing variants are `lateral`.

Contracts therefore carry `model=` and `implementation=` as separate axes: a contract has models, and a model has implementations. A specification names them with separate keys, and naming one where the other is meant is refused rather than silently accepted. Two consequences follow. A mechanism claim becomes attributable, because the record names the model it is about. And a search can distinguish an edit that changes the biology from one that changes only the arithmetic — which is the difference between an experiment and a control.

Selection is by name: `get_operator(name, model, implementation)` returns the chosen class (the defaults when none are named), and `get_contract(name)` exposes the whole contract. The registry therefore separates biological semantics from numerical realization: new numerical methods are new registered implementations, new hypotheses are new registered models, and neither is an edit to the execution engine.

A.5 Schedules

A biological model is executed by evaluating an ordered Schedule of operator applications.

During each simulation step

1. operators are evaluated in Schedule order,
2. each dynamics operator returns a per-set delta (or a field/relation/structural operator mutates the grid, the edge set, or the membership in place),
3. the engine sums the deltas acting on each set,
4. each set is integrated once, in the order its operators declare (`EMIT`: first-order `velocity` or second-order `acceleration`).

Because operators are pure and nothing is written in place except the explicitly allowed buffers, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The Schedule therefore implements the operational semantics of the main text while remaining independent of the particular numerical implementations it composes.

A.6 Model specification

Biological models are described declaratively rather than procedurally, as a single `spec.yaml` with blocks `general` / `sets` / `fields` / `operators` / `schedule`; `general` optionally carries `units:`, the run's physical scale (see below). An operator line names the contract (`op`) and, optionally, which implementation to run (`implementation:`); omitting it selects the contract's default, so a single-implementation spec is written exactly as before.

A specification therefore defines

- biological Levels and their state schemas,
- Fields,
- the operators to instantiate, each with its selected implementation,
- the execution Schedule.

Before execution the specification is validated against the registered contracts: an unknown operator, an unknown `implementation` for a known operator, a missing required parameter, a selector on an undeclared set/field, or an unresolved schedule token is rejected up front — so a spec that loads is runnable, and the capability checks (e.g. `SUPPORTED_DIMS`) run against the implementation that will actually execute.

A.7 Units, and what a dimensionless model may claim

A Plexus model computes in simulation units: the world is a unit box, `dt` is a number, a stiffness is a number. That is sufficient for a *mechanism* and insufficient for a *claim*, and the gap between the two is where quiet errors live — a membrane thickness that turns out to be 24× too large once the box is calibrated, a modulus compared against a measurement in megapascals, a turnover time whose being "four hours" rests on an unstated minutes-per-frame. None of those is caught by any validity check, because each is arithmetic that is internally consistent and externally meaningless.

The scale is therefore *declared*, once, under `general.units:`, and never inferred:

- `length_um` — micrometres per simulation length unit (the world box is 1 unit),
- `time_s` — seconds per simulation time unit; **the default is 1.0, i.e. the convention is that `dt` is in seconds**,
- `force_nN` — nanonewtons per simulation force unit; optional, and without it only force *ratios* are meaningful,
- `amount` — an optional label for a field's concentration, which is a fourth base unit that nothing mechanical needs.

Three base scales, because mechanics needs three. **Everything else is derived, and declaring a derived scale is rejected at load rather than accepted as a convenience**, since a second declaration is a second chance to disagree with the first:

quantity	dimension	from the base scales
position, radius, rest length, thickness	L	`length_um`
area / volume	L^2 / L^3	`length_um`^{2} / ^{3}
velocity	L/T	`length_um` / `time_s`
rate, $1/\tau$	$1/T$	1/`time_s`
force	F	`force_nN`
stress, Young's modulus	F/L^2	`force_nN` / `length_um`^{2} (→ Pa)
2D (membrane) modulus $Y_2 = ET$	F/L	`force_nN` / `length_um` (→ N/m)
energy	$F\,L$	`force_nN` · `length_um`
mobility $1/\gamma$	$L/(F\,T)$	derived

Two design decisions follow, and both are deliberate omissions. There is **no automatic conversion of state buffers and no dimensional checking of operator arithmetic**: operators compute in simulation units exactly as before, and `units` is a declaration attached to the run so that what comes *out* can be reported in micrometres and seconds, and so that a spec making a physical claim can be asked what it is claiming it in. Unit-checked arithmetic would be a large change and a false comfort, since the errors above are all dimensionally consistent.

And a model with no `units:` block is **dimensionless**: it loads, it runs, and the loader prints *"units: NONE DECLARED — this run is dimensionless and no result from it may be quoted with a unit"*. That is the honest state of a mechanism study, it is the state of every spec written before this contract existed, and saying it out loud at build time is the point — "dimensionless" should be a property the model states, not an oversight a reader discovers later from a number that looked like micrometres.

A.8 Reference architecture

The reference implementation follows a strict separation of concerns.

- The language (`sets` / `states` / `fields` / `operators`) defines biological semantics.
- The registry stores each operator as an `OperatorContract` with its implementations.
- Implementations (`Operator` subclasses) realize those contracts numerically.
- The validator (`schema.py`) admits only well-formed, runnable specs.
- The execution engine (`engine.py`) builds the `Hierarchy` and iterates the Schedule; it never contains mechanism-specific code.

A single audit (`tools/audit_operator_registry.py`) enforces the contract across the whole registry — every operator has a valid `family` and `kind`, a canonical `EMIT`, and a `SUPPORTED_DIMS`$\subseteq \{2,3\}$ — so the vocabulary cannot silently drift. Consequently, extending Plexus never requires modifying the execution engine: new biological mechanisms are new operator contracts, and new numerical methods are additional implementations of existing operators.

The reference implementation therefore mirrors the architecture proposed in the main text: the language remains stable while implementations and numerical backends evolve independently.

B Building the Plexus operator atlas

One of the principal objectives of the Plexus research program is to construct a comprehensive atlas of biological operators by systematically analysing the existing corpus of biological simulation models. Rather than cataloguing simulators, the atlas catalogues reusable biological mechanisms together with their numerical implementations.

Decomposition is not the objective; it is the first half of one. Taking PhysiCell, Tyssue, a material-point solver or a neural simulator apart into operators, entities and relations is worth doing because those pieces can then be put back together in combinations none of the original frameworks could express — a growing cell tissue that deforms a basement membrane embedded in an extracellular matrix, with molecular mechanisms regulating the interface, is exactly such a recomposition, and none of its parts had to be rewritten in another part's terms. The atlas is therefore read as

$$\text{existing models} \longrightarrow \text{common pieces} \longrightarrow \text{new biological compositions,}$$

with the second arrow, not the first, as the point of the exercise.

This atlas serves two complementary purposes. First, it provides the biological vocabulary from which new models are assembled. Second, it defines the search space explored by the agentic discovery framework described in the main text.

B.1 From simulators to operators

Existing simulation frameworks are not imported directly into Plexus. Instead, each framework is systematically decomposed into the language introduced in this paper.

The decomposition proceeds through the following steps:

1. identify the biological sets,
2. identify the associated state variables,
3. identify continuous fields,

4. identify biological operators,
5. identify the maps required by each operator,
6. separate biological semantics from numerical implementations,
7. register each operator in the Plexus language.

The result is an implementation-independent description expressed entirely in terms of operators, sets, states and fields.

B.2 Normalization

Different simulators frequently implement identical biological mechanisms using very different numerical algorithms. The atlas therefore groups operators according to biological semantics rather than implementation.

For example,

Biological operator	Possible implementations	Example frameworks
Mechanical interaction	MPM, FEM, Vertex, Phase Field	Taichi, Chaste, Tissue Forge
Chemotaxis	Finite differences, Neural fields	Morpheus, CompuCell3D
Electrical propagation	Hodgkin–Huxley, Neural ODE, GNN	NEURON, Jaxley, BrainPy
Diffusion	Finite differences, FEM	Morpheus, VirtualLeaf
Cell division	Vertex, CPM, Agent-based	Chaste, CompuCell3D

The objective is therefore not to catalogue implementations but to identify the smallest reusable vocabulary capable of reconstructing the existing simulation literature.

B.3 Operator metadata

Every entry in the atlas records

- the operator name, family and kind,
- its typed signature (input/output sets, state read/written, maps),
- its implementations and their capabilities (supported dimensions, differentiability),
- its biological interpretation (`MECHANISM_TAGS`) and the role of each tunable parameter (`PARAM_ROLES`),
- originating publications and reference repositories,
- validation status.

The current registry already carries most of this per operator — name, family, kind, acting set, typed signature, implementations, capabilities, mechanism tags and parameter roles, with provenance kept in the operator's docstring; the atlas adds the cross-framework publication and repository record. Consequently the atlas simultaneously documents biological mechanisms, available numerical realizations, and scientific provenance.

B.4 Atlas growth

The atlas is intended to evolve continuously.

Every newly analysed simulator contributes either

1. an implementation of an existing operator,
2. a refinement of an existing typed signature,
3. or a genuinely new biological operator.

The first outcome strengthens numerical diversity. The second improves the language. The third expands the biological vocabulary itself.

B.5 Validation of the operator algebra

The atlas provides a direct experimental test of the central hypothesis of Plexus.

If the decomposition of a sufficiently broad collection of biological simulation frameworks converges toward a compact and reusable operator vocabulary, then the proposed operator algebra constitutes a meaningful intermediate representation for biological modelling.

Conversely, repeated discovery of genuinely new operator families indicates that the language remains incomplete and must be extended.

The operator atlas therefore serves both as a repository of biological knowledge and as the principal mechanism through which the Plexus language itself evolves.

C Implementing new operators

The Plexus language grows by introducing new biological operators rather than by modifying the execution engine. A new operator should be introduced only when an existing operator cannot express the mechanism without changing its biological meaning. New solvers, discretizations, parameterizations or hardware targets should instead be added as alternative implementations of an existing operator.

C.1 Define the biological mechanism

Before implementation, the mechanism must be described independently of its numerical realization. This description should specify

- the biological transformation represented by the operator,
- its operator family,
- its input and output sets or fields,
- the state variables it reads and produces,
- the maps required to relate the participating objects.

Together, these elements define the operator's typed signature, which serves as its biological contract.

C.2 Separate semantics from implementation

The operator contract and its numerical implementations are represented separately.

A single biological operator may admit multiple implementations based, for example, on Material Point Methods, finite elements, vertex models, phase fields, Neural ODEs or graph neural networks. These implementations may differ in their numerical assumptions, spatial representation, supported dimensions or differentiability while preserving the same biological semantics.

Conversely, mechanisms should not be merged merely because they share similar code or numerical utilities. Distinct biological transformations require distinct operator contracts.

C.3 Register the operator and its implementations

Once defined, the operator is added to the registry by decorating its implementation class with `@register_operator(name, set=, kind=, family=, implementation=)` (Listing 1). The decorator creates the contract on the first implementation of a name and records

- the canonical operator name (plus any aliases), family and kind,
- its typed signature, declared as the class attributes `INPUTS` / `OUTPUTS` / `READS` / `WRITES` / `MAPS`,
- the available implementations, keyed by `implementation` name,
- their capabilities (`SUPPORTED_DIMS`, `DIFFERENTIABLE`),
- their scientific provenance (in the class docstring).

A second numerical method for the same mechanism reuses the same `name` with a new `implementation=` and is checked to share the contract's `kind`. Registration makes the mechanism available to every compatible Plexus model without introducing biological knowledge into the execution engine.

C.4 Validation

Every implementation should satisfy four requirements:

1. it satisfies the declared typed signature;
2. it reproduces the intended biological mechanism;
3. its numerical behaviour is verified;
4. its scientific provenance is preserved.

In the reference implementation the first three are gated concretely: the registry audit (`audit_operator_registry.`
checks the declared contract, schema validation checks that a spec using the operator loads, and a minimal live spec plus the test suite check its numerical behaviour.

Validation establishes structural and numerical consistency, but does not by itself establish biological validity. The latter must be assessed through comparison with observations, known limits, ablations or the originating model.

C.5 The note as a procedure: gates, and what a gate can be evidence of

The `prototype/ecm` study converged on a working form for that last paragraph, and it is worth stating as a procedure rather than as a habit. Each addition to the framework — an entity, a relation, an interface — is written up in a note with exactly four parts, and the note is *the unit of work*: nothing is considered added until it exists.

1. **Mechanism.** The biology, coarsely, with the one or two measured ratios that fix its proportions — and the arithmetic that decides what cannot be represented at all. A basement membrane is 100 nm on a 200 μm spheroid; resolving it on a grid needs 3500^3 cells, so it is a surface and its thickness is a parameter. That paragraph is what makes the rest inevitable.

2. **Equations and symbols.** The mechanism as equations, with a symbol table saying what each quantity is *of*. Intensive and extensive are distinguished here, because a feedback whose drive is extensive is ill-posed under re-meshing and the error is invisible at runtime.

3. **The Plexus2 container.** Sets, their kinds, the state each carries, the operators and their families, and the schedule — specifically which operators go inside a `substep_dt` block and which do not.

4. **Gates.** A numbered table: for each, a threshold decided *before* the run and the measured value, coloured by outcome. A threshold chosen after seeing the number is not a threshold.

Not every gate is evidence of the same kind, and conflating them is the failure mode this form exists to prevent. Across the five notes the gates fall into three tiers, and only the third is a statement about biology:

tier	asks	example, from `prototype/ecm`
bookkeeping	does the code do what the operator says?	the contact's reaction sums to zero at 1.5 × refining a loaded sheet leaves λ, area an ergy bit-identical; a re-keying operator pe no trajectory
closed form	does the implementation reproduce the physics it was *given*?	the block's ringing at 12.9 Hz against th Hz of a free–free bar; the matrix fitting a spherical shell's $C_1 r + C_2/r^2$ at $R^2 = 0.9$ effective Poisson ratio of 0.216 against it Lamé pair's 0.20
measurement	does the model agree with something *observed in cells*?	collagen IV turnover of 3–10 h; hemidesm spacing of $\sim 7T$; an epithelial cell of 8.5 collagen gel at 10–1000 Pa

The first two tiers are *verification*: they can be run before any biology is claimed, they are cheap, and they catch the errors that are otherwise invisible — a positional update that launders deformation, a spatial lookup that silently misses faces near a pole, a force applied once per substep where it should be applied once per step. The third tier is *validation*, and it is the only one that can be wrong about the world.

So the honest answer to "are these biophysics tests" is: a minority of them are, and the proportion is worth reporting. In the `ecm` study roughly two thirds of the gates are bookkeeping or closed form. That is not a deficiency — a mechanism whose implementation is unverified cannot be compared with an observation at all — but a note whose gate table is entirely first-tier has established that its code is self-consistent and nothing more.

And the third tier is only available once the model declares its units (§A.7). Before a `units:` block, a comparison with a measurement is a comparison between a number and a quantity, which is how a membrane came to be 24× too thick and a modulus came to be quoted as a pressure. With one, every measurement-tier gate becomes a conversion with a literature interval around it, and it can fail: in the `ecm` study, declaring the scale turned a contact penetration of "0.82 grid cells" — which sounds small, and was reported as an improvement — into 15 μm, nearly two cell diameters, which is the strongest single argument for replacing the penalty contact with a barrier formulation.

And that failure did not open a new gate — it showed an existing one was denominated wrongly. The same measurement was already gated: *penetration stays bounded, threshold* < 2 *grid cells*, passing comfortably at 0.82. A threshold in grid cells is a threshold in the currency of the discretisation, which is the currency in which it is easiest to pass, and a green row of that kind reads as an endorsement the interface has not earned. Both questions are worth asking — does the penalty run away, and is the surface a surface — but only the second is about the model. So the rule the units block yields is not "add a units gate" but

> *a gate's threshold belongs in the unit of the phenomenon, not of the mesh.*

Applying it is cheap and mostly retrospective: it changed two rows in the `ecm` notes and left the rest, because a threshold already expressed as a ratio, a residual against machine precision, or a fraction of a control is dimensionless and was never at risk.

C.6 Promotion

Prototype implementations should not enter the core registry directly. Promotion requires a stable operator contract, at least one validated implementation, clear scientific provenance, and evidence that the mechanism is reusable beyond its originating prototype.

The guiding principle is

> *Prototype freely; promote biological mechanisms conservatively.*

D Refactoring guidelines

The current Plexus repository predates the formalization of the language presented in this paper. Consequently, much of the biological semantics is embedded directly in numerical implementations. Refactoring aims to recover these semantics and progressively reorganize the codebase around the language abstractions introduced in this work.

D.1 General principles

Every refactoring should follow the same principles.

- Preserve biological behaviour.
- Improve separation between biological semantics and numerical methods.
- Replace implicit assumptions by explicit typed signatures.
- Prefer extending the language rather than adding special cases.
- Refactor incrementally while preserving existing functionality.

D.2 Recommended workflow

When refactoring an existing module, contributors should proceed in the following order.

1. Identify the biological mechanism implemented by the code.

2. Determine whether it corresponds to an existing operator.
3. If not, introduce a new operator contract.
4. Separate the biological contract from the numerical implementation.
5. Register the implementation.
6. Validate that the refactored implementation reproduces the original behaviour.

D.3 Code migration

Existing code should progressively migrate toward the following organization.

- Biological concepts belong to operator contracts.
- Numerical algorithms belong to implementations.
- Execution logic belongs to the runtime.
- Data structures belong to Levels, Fields and Hierarchies.

The execution engine should never contain mechanism-specific code.

D.4 Incremental evolution

Refactoring is expected to be continuous. During the transition, legacy modules and language-native modules may coexist. New developments should preferentially follow the language architecture, while legacy components are progressively decomposed into reusable operators as they are revisited.

The objective is not to rewrite the repository, but to allow the language to emerge progressively from the existing implementation.

E Reference corpus for the operator atlas

The Plexus operator atlas is constructed by systematically decomposing existing simulation frameworks into biological operators, typed signatures and numerical implementations. To ensure reproducibility, the initial atlas is built from open-source projects for which both the scientific publication and an implementation are publicly available.

Rather than treating each simulator as an indivisible software package, every framework is analysed according to the methodology described in Appendix B. The objective is to recover the reusable biological mechanisms implemented by each project and register them as operators within the Plexus language.

The reference corpus is organized into several thematic categories.

E.1 Cell and tissue mechanics

These frameworks describe multicellular mechanics, growth, adhesion, proliferation, remodelling and morphogenesis.

- Chaste

- CompuCell3D
- Morpheus
- Tissue Forge
- VirtualLeaf
- PhysiCell
- Active Vertex Model for Cell-Resolution Description of Epithelial Tissue Mechanics
- Multicellular Simulations with Shape and Volume Constraints using Optimal Transport
- Cell Simulation as Cell Segmentation

Typical operators extracted from these models include

- mechanical interaction,
- adhesion,
- active stress,
- cell growth,
- cell division,
- apoptosis,
- neighbour rewiring,
- particle–cell aggregation,
- optimal transport.

E.2 Vertex models and 3D morphogenesis

Feynman's second blackboard line urges not just creation but the understanding of *everything that has already been built*. Reproducing a *known* result — Okuda *et al.*'s autonomous 3D tubulation and branching, a Turing reaction–diffusion coupled to a 3D vertex model — inside Plexus therefore begins by surveying, as exhaustively as we can, the vertex-model and morphogenesis literature it descends from and the code that ships with it. This focused sub-corpus is that survey.

The Okuda lineage (the reproduction target).

- Combining Turing and 3D vertex models reproduces autonomous multicellular morphogenesis with undulation, tubulation, and branching — Okuda et al. (2018), the target.
- Three-dimensional vertex model for simulating multicellular morphogenesis — Okuda et al. (2015), the base 3D vertex model (volume/surface energy, reversible network reconnection).
- Reversible network reconnection model for simulating large deformation in dynamic tissue morphogenesis — Okuda et al. (2013), the RNR / T1 topology operator.

Vertex-model frameworks that ship code.

- tyssue — the true-vertex (AVM) Python framework; the mechanical substrate of our prototype.
- tvm (Zhang & Schwarz, 2022; code) — a maintained 3D space-filling vertex model, a numerical-validation target.
- Graph vertex model (Sarkar & Krajnc, 2024; code neoVM) — topology as a graph; 3D reconnection decomposed into elementary T1 moves.
- 3D-Vertex-Model — a Mathematica port of Okuda's 2012/2013 reconnection model.
- Tissue Forge (2023) — a general C++/Python vertex-modelling substrate.

Differentiable vertex / morphogenesis models (nearest to the Plexus architecture).

- VertAX (Pasqui et al., 2026; code) — a differentiable JAX vertex model; benchmarks implicit differentiation through equilibrium against an unrolled relaxation loop.
- jax-morph (Deshpande et al., 2025; code) — composable differentiable growth / division / diffusion modules, the closest existing analogue of an operator registry.
- Differentiable Design for Morphogenesis (Beker & Dumitrascu, 2026) — a differentiable forward tissue model with a GNN-based inverse.
- Mean-field elastic moduli of a 3D cell-based vertex model (Kim, Zhang & Schwarz, 2024) — closed-form moduli to validate a 3D implementation against.

Reaction–diffusion on growing domains (the dilution term the patterning must carry).

- Turing patterns on a two-component isotropic growing system, Part 1 (Ledesma-Durán, 2023) — growth adds a dilution term that makes the homogeneous base state time-dependent.
- Base state of growing reaction-dilution systems (Ledesma-Durán, 2026) — how patterning conditions change once reaction rates must balance local volume change.

Tube, lumen and sheet morphogenesis (the target morphology).

- Three-dimensional morphogenesis of epithelial tubes (Yu & Li, 2024).
- Mechanochemical dynamics in multicellular lumens (Yu et al., 2024).
- A virtual spherical-shell multicellular platform (Fujiwara et al., 2022) — the same shell geometry we build on.
- Growth and shrinkage of tissue sheets on substrates: buds, buckles, and pores (Noguchi & Elgeti, 2024).
- Computational approaches for simulating luminogenesis (Fuji et al., 2022).
- Modeling polarity-driven laminar patterns in bilayer tissues (Moore, Dale & Woolley, 2023).

Surveys and further corpus.

- No country for old frameworks? Vertex models and their ongoing reinvention (Briñas-Pascual et al., 2024).
- Cell-based computational models of organoids: a systematic review (Neagu et al., 2026).
- Beads, springs and fields in cell biophysics (Sorichetti et al., 2026).
- Modelling cellular interactions and dynamics during kidney morphogenesis (Cook et al., 2022).
- Avalanches during epithelial tissue growth: a Drosophila eye-disc model (Courcoubetis et al., 2022).

E.3 Reaction–diffusion and signalling

These models describe transport, biochemical signalling and spatial pattern formation.

Representative projects include

- Morpheus
- Reaction-Diffusion Systems in Intracellular Transport and Control
- A Programmable Reaction-Diffusion System for Spatiotemporal Cell Signalling Circuit Design
- SPDEs

Representative operators include

- diffusion,
- reaction,
- secretion,
- uptake,
- field sampling,
- broadcast signalling.

E.4 Neural circuits

These frameworks model neuronal signalling and connectome dynamics.

Representative projects include

- Jaxley
- BrainPy
- Connectome-GNN

Typical operators include

- synaptic transmission,
- membrane integration,
- plasticity,

- neuromodulation.

E.5 Mechanobiology and collective behaviour

Several recent frameworks couple mechanics with biochemical signalling and collective migration.

Representative examples include

- ERK-mediated mechanochemical wave models
- Active Vertex mechanics
- Collective cell migration models
- Bioelectric morphogenesis

Representative operators include

- mechanochemical coupling,
- force transmission,
- wave propagation,
- chemotaxis,
- collective polarity.

E.6 Inverse modelling and differentiable simulation

These projects are particularly relevant for the mechanistic inverse modelling described in Section 6 because they expose differentiable representations of biological systems.

Representative projects include

- MultiCell: Geometric Learning in Multicellular Development
- Differentiable Rendering for 3D Fluorescence Microscopy
- Inferring Cell Junction Tension and Pressure from Cell Geometry
- Machine Learning Interpretable Models of Cell Mechanics from Protein Images
- 3D Single-Cell Shape Analysis using Geometric Deep Learning

Typical operators include

- image formation,
- geometric inference,
- parameter estimation,
- mechanical inversion,
- differentiable observation.

E.7 Image analysis and quantitative biology

These projects provide computational operators that transform experimental observations into structured biological measurements.

Representative projects include

- TissueMiner
- ERNet
- DeXtrusion
- FlowShape
- CellRank

Representative operators include

- segmentation,
- tracking,
- graph construction,
- feature extraction,
- trajectory inference.

E.8 Interactive artificial-life programs

Alongside the research-grade simulation frameworks above, a broad ecosystem of interactive artificial-life and complex-systems programs explores the same emergent mechanisms — self-organization, growth, pattern formation and evolution — often in real time and in the browser. These provide an accessible, complementary corpus for the atlas:

- Lenia
- Evoloops
- Particle Life
- Bibites
- Life Engine
- Convolutional CA
- Game of Life
- Multi-neighborhood CA
- Alien Project
- Germs genetic algorithm
- Subnautica
- Synthetic Selection
- Neural CA

- Particle Lenia

E.9 Building the atlas

Each framework in the reference corpus is analysed using the same procedure.

1. Identify the biological sets.
2. Identify their state variables.
3. Identify continuous fields.
4. Identify the biological operators.
5. Identify the maps relating the participating objects.
6. Separate biological semantics from numerical implementation.
7. Register the resulting operators in the Plexus atlas.

The objective is not to reproduce the original software architecture, but to recover the reusable biological mechanisms that underlie existing simulators. Each analysed framework may contribute additional implementations of existing operators, refine existing typed signatures, or reveal genuinely new biological operators. Consequently, the atlas evolves continuously as new open-source simulation frameworks become available.

F An extended repository survey

Appendix E lists the initial reference corpus. This appendix expands it into a broader candidate survey of fifty open-source repositories, grouped into five mechanism classes. For each repository we record its primary model family and the operator families we expect to extract from it. These operator families are candidates to be confirmed by source-level inspection following the methodology of Appendix B: the entries reflect what each project advertises, not yet a normalized set of registered contracts.

F.1 A. Multicellular and agent-based dynamics

These are likely the highest-yield sources for cell states, cell–cell mechanics, phenotype transitions, growth, division, death, secretion, uptake and agent–field coupling. PhysiCell, for example, targets large 2-D/3-D multicellular systems, while PhysiBoSS adds intracellular Boolean signalling.

#	Repository	Primary model family	Likely operator families
1	MathCancer/PhysiCell	Off-lattice cell agents	divide, die, adhere, repel, migrate, secrete, uptake
2	PhysiBoSS/PhysiBoSS	PhysiCell + Boolean networks	all above + activate, inhibit, switch phenotype
3	MathCancer/BioFVM	Diffusive microenvironment	diffuse, decay, source, sink, Dirichlet boundary
4	Chaste/Chaste	Multiscale cell/tissue simulation	cell cycle, forces, PDE coupling, electrophysiology
5	CompuCell3D/CompuCell3D	Cellular Potts	copy, adhere, constrain volume, chemotax, divide
6	BioDynaMo/biodynamo	General biological agents	grow, divide, move, interact, diffuse signals
7	tissue-forge/tissue-forge	Particle biology and chemistry	force, bond, react, diffuse, secrete, assemble
8	HaseloffLab/CellModeller	Growing microbial cells	elongate, divide, collide, signal, regulate
9	RobertGregg/CellularPotts.jl	Cellular Potts in Julia	adhere, move, divide, constrain area/volume
10	ingewortel/artistoo	Browser Cellular Potts	copy, adhesion, volume constraint, chemotaxis
11	mathbioleiden/Tissue-Simulation-Toolkit	2-D Cellular Potts	adhesion, motility, PDE signalling, growth
12	shabaz/gpu-cpm	GPU Cellular Potts	lattice copy, adhesion energy, volume constraint

CompuCell3D and Artistoo are explicitly Cellular Potts environments; the Tissue Simulation Toolkit is another 2-D Glazier–Graner implementation.

F.2 B. Vertex, Voronoi and epithelial mechanics

These repositories should yield topology-changing and geometric operators that agent-centre models often hide: area and perimeter elasticity, junction tension, T1 transitions, cell extrusion, polygon division and boundary remodelling. CellGPU covers Voronoi and dynamic vertex models, while Tyssue represents epithelial sheets.

#	Repository	Primary model family	Likely operator families
13	sussmanLab/cellGPU	Voronoi/SPV and vertex	area elasticity, perimeter tension, self-propel, T1
14	DamCB/tyssue	Epithelial vertex model	contract, stretch, divide, extrude, reconnect
15	Pablo1990/pyVertexModel	3-D vertex/scutoid	apical/basal tension, lateral adhesion, topology
16	yasuhiroinoue/PyCellVertex	2-D vertex model	growth, division, rearrangement, morphogenesis
17	mmarinriera/VertexModel	Epithelial vertex dynamics	area force, edge tension, neighbor exchange
18	mshagirov/vertex_simulation	Cell-monolayer vertex dynamics	relax vertices, constrain boundary, compute energy
19	kylechanphy/TissueSim	Active vertex model	self-propel, align, deform, rearrange
20	vmansov/clustering_vmodel	Mutant-cell vertex model	cluster, alter mechanics, relax mesh
21	lixinzhi4429/VM_tissue_growth	Growing epithelial vertex model	grow, divide, stress-feedback, align polarity
22	BanerjeeLab/HNP_model	Developmental vertex model	crawl, tension, stress alignment, feedback

The Python vertex implementations explicitly advertise tissue morphogenesis, rearrangements, area/perimeter geometry and boundary constraints.

F.3 C. Spatial molecular and transport processes

This family is essential because it yields operators below the cell scale: Brownian motion, binding, unbinding, unimolecular and bimolecular reactions, membrane crossing, adsorption, desorption, compartment transport and surface diffusion. Smoldyn, ReaDDy and MCell are complementary particle-based spatial simulators; VCell spans deterministic, stochastic, particle and moving-boundary models.

#	Repository	Primary model family	Likely operator families
23	readdy/readdy	Particle reaction–diffusion	diffuse, react, bind, unbind, transform topology
24	ssandrews/Smoldyn	Brownian molecular particles	diffuse, collide, react, adsorb, cross membrane
25	mcellteam/mcell	Monte Carlo cellular microphysiology	diffuse, surface-react, release, absorb
26	mcellteam/cellblender	MCell geometry/-model authoring	define compartments, membranes, sources and reactions
27	CNS-OIST/STEPS	Stochastic reaction–diffusion	react, diffuse, channel flux, membrane current
28	virtualcell/vcell	Spatial systems biology	ODE, reaction–diffusion, advection, electrophysiology
29	GollyGang/ready	Cellular automata/PDE exploration	local update, diffuse, react, mesh evolution
30	spatial-model-editor/ spatial-model-editor	Spatial SBML models	compartmentalize, react, diffuse, impose geometry

F.4 D. Intracellular signalling and biochemical networks

These projects can supply operators for reaction networks, rule-based molecular interactions, state transitions, stochastic events, conservation laws and parameterized kinetic laws. VCell and COPASI are foundational mechanistic cell-biology platforms, while BioNetGen describes rule-based interactions without enumerating every molecular species.

#	Repository	Primary model family	Likely operator families
31	copasi/COPASI	Biochemical reaction networks	react, catalyze, inhibit, transport, event
32	RuleWorld/bionetgen	Rule-based biochemical models	bind, unbind, modify, synthesize, degrade
33	sys-bio/tellurium	Reproducible systems biology	integrate ODE, simulate events, compose models
34	sys-bio/roadrunner	SBML simulation engine	integrate kinetics, steady state, sensitivity
35	pysb/pysb	Programmatic rule-based modelling	bind, catalyze, translocate, express, degrade
36	GillesPy2/GillesPy2	Stochastic biochemical kinetics	stochastic reaction, propensity, event
37	StochSS/StochSS	Stochastic systems biology	simulate ensembles, spatial reactions, parameter sweep
38	AMICI-dev/AMICI	Differentiable ODE models	integrate, differentiate, infer parameters
39	opencobra/cobrapy	Constraint-based metabolism	flux balance, uptake, secrete, constrain reaction
40	BhallaLab/moose-core	Multiscale cellular simulation	kinetics, diffusion, electrophysiology, signalling

F.5 E. Neural computation as cellular simulation

Although these are neuroscience frameworks, they are highly relevant to Plexus because they expose another mature corpus of reusable cell-level operators: membrane integration, ion-channel gating, axial conduction, synaptic transmission, spike generation, delays and plasticity. Chaste itself also combines electrophysiology with tissue and cell modelling.

#	Repository	Primary model family	Likely operator families
41	neuronsimulator/nrn	Multicompartment neurons	integrate voltage, gate channels, axial current, spike
42	brian-team/brian2	Equation-defined neural networks	integrate state, threshold, reset, synapse
43	nest/nest-simulator	Large spiking networks	spike, transmit, delay, plasticity
44	arbor-sim/arbor	HPC multicompartment neurons	cable solve, channel current, synaptic event
45	AllenInstitute/bmtk	Brain network models	construct/connect networks, drive, record
46	Neurosim-lab/netpyne	NEURON network specification	instantiate cells, connect, stimulate, simulate
47	LFPy/LFPy	Biophysical neurons and fields	membrane current, extracellular field, electrode sample
48	jonescompneurolab/hnn-core	Cortical circuit biophysics	synaptic drive, dendritic integration, field generation
49	neurolib-dev/neurolib	Neural mass/whole-brain dynamics	local dynamics, network coupling, delayed propagation
50	NeuralEnsemble/Nengo	Neural dynamical systems	encode, transform, filter, learn

F.6 A preliminary atlas record

For each repository the atlas stores a single record separating the raw mechanisms named by the project from their normalized Plexus contracts, together with the evidence and validation status:

Listing 2: One atlas record per repository: raw mechanisms are separated from their normalized Plexus contracts, with evidence and validation status.

```
repository: MathCancer/PhysiCell
paper: DOI-or-PMID
model_family: off_lattice_multicellular
scale:
  - cell
  - tissue
sets:
  - cell
  - substrate_voxel
fields:
  - oxygen
  - signalling_substrate
maps:
  - cell_cell_neighborhood
  - cell_voxel_sampling
operators_raw:
  - cell_cell_force
  - phenotype_update
  - cell_division
```

```
  - apoptosis
  - substrate_secretion
  - substrate_uptake
operators_normalized:
  - repel
  - adhere
  - update_phenotype
  - divide
  - die
  - produce
  - consume
evidence:
  paper_section: ...
  code_path: ...
status: candidate | inspected | normalized | implemented
```

F.7 What this corpus should reveal

The fifty repositories are not fifty independent vocabularies. They repeatedly implement the same mechanisms under different names and numerical schemes. A preliminary normalization should therefore distinguish the biological mechanism from its numerical implementation while preserving a single operator contract. For instance, cell division appears as

- **Mechanism:** `divide`.
- **Implementations:** vertex split, CPM mitosis, off-lattice daughter insertion.
- **Contract:** one cell state and geometry → two valid daughter-cell states and an updated neighbourhood.

Likewise, `diffuse` should remain one operator contract even when implemented by finite differences, finite volumes, stochastic particles or a neural implicit field.

Source-level extraction would begin with twelve high-yield representatives — PhysiCell, CompuCell3D, Chaste, Tissue Forge, BioDynaMo, CellGPU, Tyssue, Artistoo, ReaDDy, Smoldyn, VCell and NEURON. Together they span most anticipated Plexus relation classes: set → set, set → field, field → set, field → field, topology change, containment and cross-scale coupling. The list is broad enough to begin measuring operator *saturation*: after each repository we record how many genuinely new normalized contracts appear versus aliases of contracts already registered. Saturation — new frameworks contributing implementations rather than new contracts — is the empirical signal that the operator algebra of the main text is complete.

G Making a cell in Plexus

A cell is not one kind of thing, and that is the difficulty. Its interior is a dense fluid, its boundary is a surface that can stretch and tear, its skeleton is a network of filaments, its nucleus is a body of its own, and its receptors are discrete switches. A simulator built for any one of these is wrong for the others, so the usual answer is to write a special-purpose cell simulator that hard-codes a compromise between them.

Plexus offers a different answer. A cell is a *parent* that contains several distinct sets, each represented in the way its own biology asks for, joined by containment and by ordinary interaction operators. Nothing about the cell is built into the engine. If this works, it answers a sharper question than whether Plexus can simulate a cell:

Can a cell be composed from heterogeneous mechanistic substrates — material, membrane, filaments, organelles and sensors — without introducing a special cell simulator?

G.1 The composition

Each part is a set inside the cell, and the substrate differs on purpose. Making them all the same material with different stiffness would demonstrate parameter variation, not composition.

Part	**Substrate**	**Operators**
cytosol	dense particle material	the material-point cycle
protein	particles carried by that material	advection, diffusion, reaction
membrane	a bonded surface network	bond, crosslink, remodel, unbond
nucleus	a second material body	the same cycle, its own material
receptor	discrete switches	bind, unbind, pull, sense

Two of these share a substrate and remain separate sets: the cytosol and the nucleus are both discretised as material points, but they are different sets with different materials and different densities. That is the distinction between composition and parameterisation.

The sets are named for what they are, not for how they are computed. A set called `mpm_particle` names a numerical method and violates the separation the main text rests on; `cytosol` names a biological object, and the material point method is how one operator chooses to discretise it.

G.2 The first experiment

One cell. The interior is a buoyant liquid holding two protein species of different density; the nucleus is denser than either; a bonded membrane keeps the cell intact; receptors sit on the membrane and read the local field.

Buoyancy drives the motion. A liquid of uniform density under gravity does nothing, so the circulation has to come from a density difference, and here it comes from two: the lighter protein species rises through the heavier one and the interior overturns, and the nucleus sediments and pushes cytosol around itself as it falls. Neither flow is imposed. Both are consequences of density in a fluid, which is the point — a prescribed stirring would prove nothing about composition.

Buoyancy belongs in the grid solve rather than in a per-particle force, for the same reason surface tension already does: it depends on the local density, and only the grid knows what a particle is displacing.

G.3 What is missing

Two mechanisms do not yet exist, and both are small next to the composition itself.

Advection. A field or a species is not currently carried by the material it sits in. Proteins would diffuse and react but not flow, which removes the one thing the experiment is for. This is a new operator.

Buoyancy. A term in the grid solve, taking the local density against a reference, in the same place and the same form as surface tension.

Everything else exists. Containment of several distinct sets under one parent already builds; the membrane operators already bond, crosslink and rupture, and need only to be attached to a cell rather than to a matrix; the receptors already bind and pull.

G.4 What would count as evidence

One measurement per substrate, not a gallery of pictures. A composition that looks right and conserves nothing has demonstrated rendering.

- mass per compartment over time — circulation must conserve it, and drift is the first sign the coupling is wrong;
- membrane bond state — how many bonds are stretched and whether any ruptured, which is the integrity claim and the one thing a merely stiffer material could never show;
- receptor occupancy over time;
- the grid alongside the cell, because if the membrane is thinner than one grid cell then its mechanics is a property of the resolution rather than of the membrane.

The last is not a formality. A previous coupling in this project was found to have its strength set by grid spacing rather than by biology, and the measurement that revealed it was a picture of the grid beside the object it was resolving.